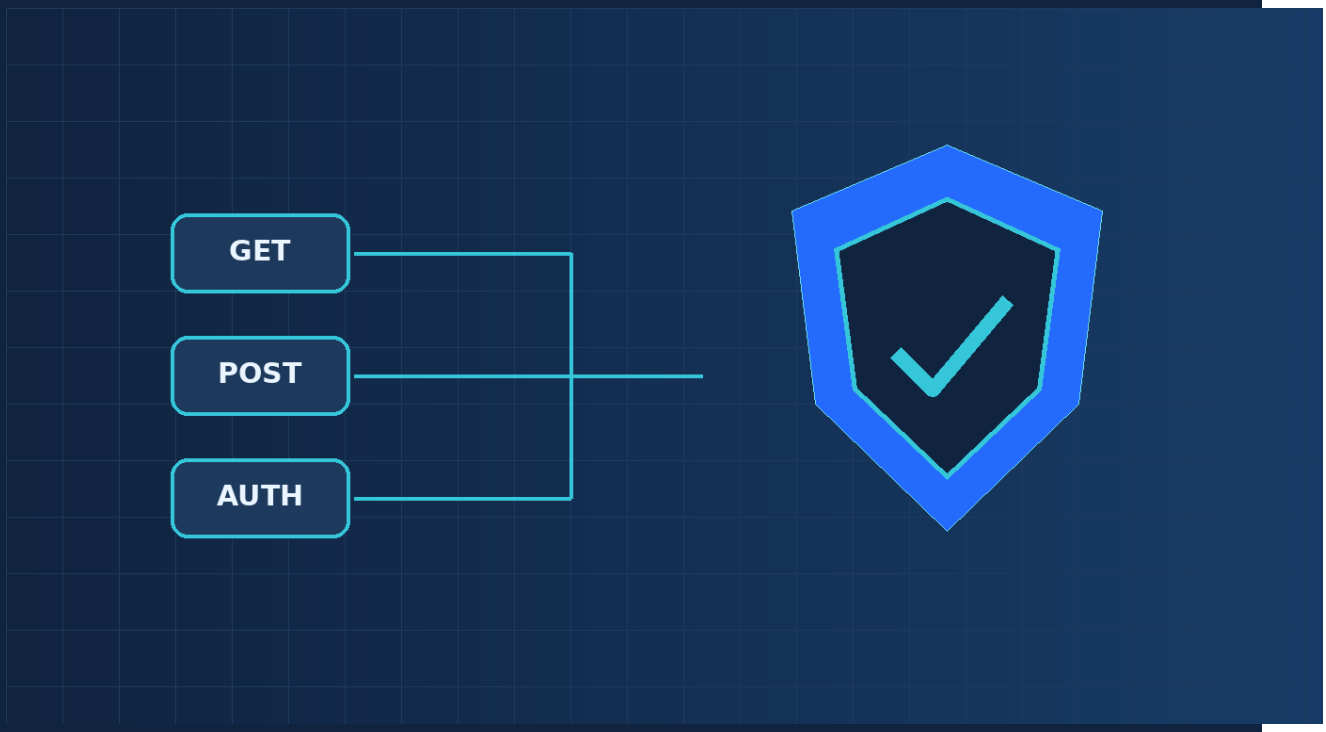



# API SECURITY RISK ANALYSIS REPORT

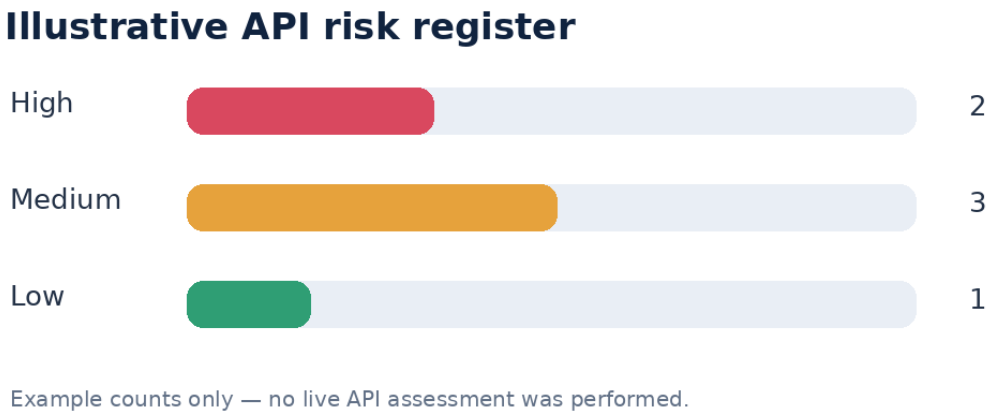
Authentication • Authorization • Data exposure • Rate limiting • Input validation

**Prepared for:** Training / portfolio demonstration
**Report date:** 16 October 2026
**Assessment target:** Fictional Demo API / approved test environment

**IMPORTANT SCOPE NOTE: This is a portfolio-ready sample. No live API was supplied and no endpoints were tested. The findings are illustrative risks to check in an authorized test environment, not verified vulnerabilities. Replace them with observed evidence before presenting the report as an actual assessment.**

# 1. Executive Summary

APIs connect applications and services and often handle sensitive data, identity, payments, and business workflows. A security review checks whether an API exposes only the data and actions a caller is permitted to access, validates inputs, protects secrets, and remains resilient under unexpected traffic.




## Business takeaway

An API can appear to work correctly while still allowing a user to access another user's records, retrieve excessive information, or send too many requests. The most important checks are authorization at every object and function, safe data handling, and clear limits on automated requests.

# 2. Scope & Rules of Engagement

**Target:** Fictional Demo API / an owner-approved sandbox
**Included:** Documented test endpoints, authentication behavior, object-level authorization, response data minimization, input validation, rate-limit behavior, and error handling.
**Excluded:** Unauthorized public targets, production data extraction, denial-of-service testing, brute force, destructive requests, and testing beyond written scope.

## Authorization first

**Use only APIs you own or are explicitly authorized to test. Prefer provider-maintained sandboxes and test accounts with synthetic data. Confirm permitted endpoints, credentials, request rates, test windows, and emergency contact. Never place API keys or bearer tokens in a public repository.**

# 3. Tools & Methodology

| Tool | Purpose | Safe-use notes |
|---|---|---|
| Postman / Insomnia | Send controlled requests, compare responses, and manage environment variables. | Use a sandbox and secret variables; never commit tokens or export them into shared collections. |
| Browser DevTools | Inspect browser network calls, response bodies, headers, and client-side errors. | Redact cookies, tokens, personal data, and private endpoints in screenshots. |
| Public demo APIs | Practice request/response inspection using documented demo endpoints. | Follow provider terms and avoid load or security tests unless explicitly permitted. |
| Docs / PDF | Record evidence, impact, severity, and remediation. | Separate verified observations from assumptions and sample findings. |

## Assessment workflow

1) Confirm scope and test accounts. 2) Review API documentation and expected roles. 3) Send baseline requests in Postman/Insomnia. 4) Compare least-privileged and authorized test-account responses. 5) Check data minimization, input validation, errors, and documented rate limits. 6) Record sanitized evidence. 7) Rate likelihood and business impact. 8) Recommend fixes and retest.

# 4. Risk Classification

| Severity | Interpretation | Typical response |
|---|---|---|
| High | Could expose sensitive records or enable unauthorized high-impact actions with realistic conditions. | Prioritize immediately; contain exposure and retest urgently. |
| Medium | Meaningful weakness with limited scope or additional prerequisites. | Schedule a near-term fix and verify with controlled tests. |
| Low | Limited direct impact or defense-in-depth gap. | Address during routine maintenance. |

# 5. Illustrative Risk Register

**The following examples describe common API weaknesses to assess. They are not confirmed findings against a real service. Add endpoint, request ID, timestamp, role, sanitized response, and reproduction steps only after validation in an approved environment.**

## API-01 — Broken Object-Level Authorization (BOLA)

HIGH

| | |
|---|---|
| **Risk** | A caller may be able to request an object by changing an identifier and receive another user's record if the server does not enforce ownership or access policy. |
| **Business impact** | Potential exposure of customer data, privacy breach, regulatory impact, and loss of trust. |
| **Evidence to collect** | Use two approved test accounts with synthetic records. Verify that each account can access only its own objects. Record status code and redacted response; do not retrieve real users' data. |
| **Remediation** | Enforce server-side authorization for every object and every request. Derive identity from the authenticated session, apply tenant/owner checks centrally, and add automated negative tests. |

## API-02 — Excessive Data Exposure

HIGH

| | |
|---|---|
| **Risk** | An endpoint may return more fields than the client needs, such as internal identifiers, personal details, or account metadata. |
| **Business impact** | Sensitive information can leak to users or apps that do not need it, increasing breach impact. |
| **Evidence to collect** | Compare the documented response schema with the actual response using synthetic data. Identify unnecessary fields without saving secrets or personal data. |
| **Remediation** | Use response DTOs/allowlists, return only required fields, classify sensitive data, and test response schemas in CI. |

| API-03 — Weak or Inconsistent Authentication | MEDIUM |
|---|---|
| **Risk** | Token validation, expiry, issuer/audience checks, or key handling may be inconsistent across endpoints. |
| **Business impact** | A stolen, expired, or incorrectly scoped credential may continue to work or grant excessive access. |
| **Evidence to collect** | In a sandbox, verify documented behavior for missing, invalid, expired, and wrong-scope credentials. Never publish token values. |
| **Remediation** | Use a trusted identity provider, validate signature/issuer/audience/expiry, rotate secrets, keep tokens short-lived, and use least-privilege scopes. |

| API-04 — Missing or Ineffective Rate Limits | MEDIUM |
|---|---|
| **Risk** | Sensitive or expensive endpoints may not have effective per-user, per-token, or per-IP limits. |
| **Business impact** | Automated abuse can increase costs, degrade service, or enable repeated guessing attempts. |
| **Evidence to collect** | Check documented limits using a small, explicitly approved request count. Observe 429 responses and Retry-After headers; do not perform load testing. |
| **Remediation** | Apply adaptive limits to authentication and expensive operations, return 429 with clear retry guidance, monitor abuse, and test limits without disrupting service. |

## API-05 — Insufficient Input Validation

MEDIUM

| | |
|---|---|
| **Risk** | The API may accept unexpected types, oversized values, or invalid fields without clear validation. |
| **Business impact** | Malformed input may cause errors, inconsistent records, or downstream processing problems. |
| **Evidence to collect** | Send a small set of harmless boundary and type-mismatch values to a sandbox endpoint; avoid payloads intended to exploit or disrupt systems. |
| **Remediation** | Validate server-side against schemas, constrain length/ranges/enums, reject unknown fields where appropriate, and use parameterized queries. |

## API-06 — Verbose Errors or Insecure Configuration

LOW

| | |
|---|---|
| **Risk** | Responses may expose stack traces, internal paths, debug details, or overly permissive CORS/security headers. |
| **Business impact** | Attackers may learn implementation details or browser-based clients may receive broader access than intended. |
| **Evidence to collect** | Review safe error responses and headers in the test environment. Compare CORS allowlists to documented client origins. |
| **Remediation** | Return generic errors with correlation IDs, keep details in protected logs, disable debug mode, and restrict CORS to trusted origins. |

# 6. Remediation Roadmap

| Priority | Action | Owner | Retest evidence |
|---|---|---|---|
| 1 — High | Enforce object-level authorization on all data access. | API / backend team | Cross-account negative tests fail safely. |
| 2 — High | Minimize response fields and review sensitive data. | API / product team | Response schema contains only approved fields. |
| 3 — Medium | Standardize token validation and scopes. | Identity / platform team | Invalid/expired/wrong-scope tokens are rejected. |
| 4 — Medium | Set safe rate limits and input schemas. | API / SRE team | Limits and validation behavior match documentation. |
| 5 — Low | Reduce debug detail and tighten CORS. | Platform team | Errors are generic; only approved origins are allowed. |

# 7. Safe Postman / Insomnia Test Checklist

- ■ Select the provider's official sandbox and read its terms.
- ■ Store tokens in local or secret-managed environment variables, not request text or shared exports.
- ■ Test missing, invalid, expired, and insufficient-scope credentials only with approved test accounts.
- ■ Compare object access across two accounts that you control and synthetic records only.
- ■ Check response fields against the documented schema.
- ■ Check validation with harmless malformed values and small boundary cases.
- ■ Confirm rate-limit behavior only within an explicitly approved request count.
- ■ Save redacted evidence, response status, timestamp, and environment; remove secrets before sharing.

# 8. Reporting & Limitations

A defensible finding should include a unique ID, endpoint and method, prerequisites, concise reproduction steps, expected versus actual behavior, sanitized evidence, impact, severity rationale, remediation, owner, and retest result. Do not include bearer tokens, API keys, customer records, or production secrets.

**Limitations: No API URL, OpenAPI specification, Postman collection, credentials, or observed responses were supplied. Therefore no endpoint was tested and no vulnerability is verified. Rate-limit behavior and access-control gaps cannot be reliably confirmed from documentation alone; they require controlled tests in an authorized environment.**

# 9. Conclusion

API security is fundamentally about ensuring each request is authenticated, authorized, validated, minimized, and monitored. Prioritize access control and sensitive data exposure, then strengthen token handling, rate limits, input validation, and error handling. Retest fixes and document evidence so technical teams and business stakeholders can agree on next steps.

### Client-friendly summary

**This sample report identifies common API risk areas and provides a safe method for validating them in an approved test environment. The highest priorities are preventing cross-user data access and limiting unnecessary data returned by endpoints. Findings should be confirmed with controlled tests before they are reported as actual vulnerabilities.**

# Appendix A — Safe example request pattern

Use only the official sandbox documented by the API provider. In Postman or Insomnia, select a request from the provider's documentation, use synthetic data, and store any required token in a secret environment variable. Begin with normal documented behavior; do not guess endpoints or send disruptive payloads.

# Appendix B — Suggested repository structure

API-Security-Risk-Analysis/
■■■ API_Security_Risk_Analysis_Report.pdf
■■■ README.md
■■■ api_risk_register.csv
■■■ assets/
■■■ api_security_cover.png
■■■ api_risk_matrix.png

Prepared as an educational portfolio artifact. Do not represent illustrative risks as verified findings from a real service.